LAPORAN PRAKTIKUM

TEKNOLOGI IOT II

Dosen Pengampu:

1. Agus Ambarwati,S.Pd., M.Kom.
2. Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T., IPM., ASEAN Eng.



Disusun Oleh:

Machmud Al Hasan (NPM 25782044)

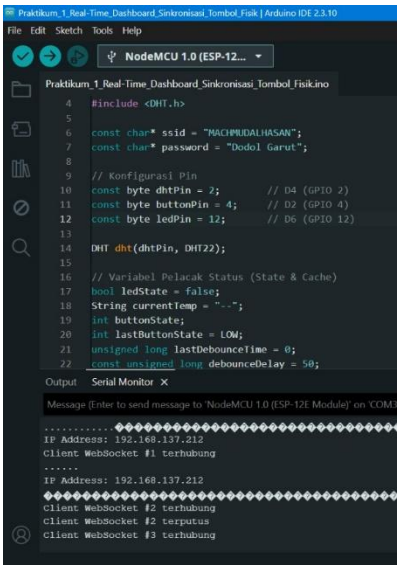
PROGRAM STUDI TEKNOLOGI REKAYASA INTERNET

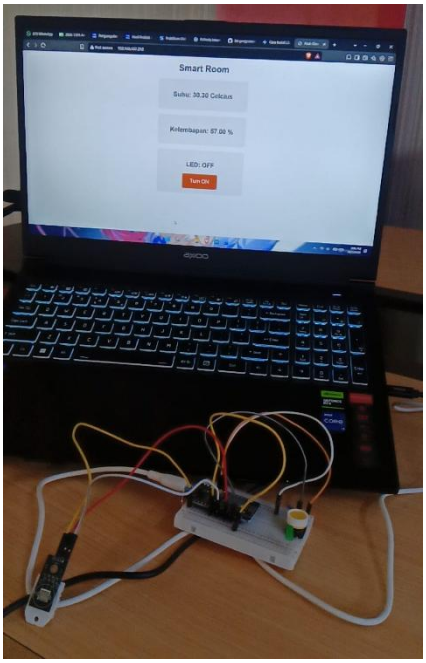
JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

TAHUN 2026

Praktikum 1: Real-Time Dashboard & Sinkronisasi Tombol Fisik





A. Langkah Kerja

1. **Perakitan Perangkat Keras (Hardware)**

   a. Merangkai lampu LED di breadboard, menghubungkan kaki panjangnya ke pin D6 NodeMCU dan kaki pendeknya ke resistor 220 Ohm yang mengarah ke pin GND.

   b. Memasang sensor DHT22. Pin VCC dihubungkan ke 3V3, pin GND ke GND, dan pin data dihubungkan ke D4.

   c. Memasang Push Button dengan rangkaian *pull-down*. Satu kaki dihubungkan ke 3V3, kaki seberangnya dihubungkan ke pin D2 sekaligus disambungkan ke resistor 10k Ohm yang menuju GND.

2. **Pemrograman Kode (Software)**

   a. Menuliskan nama jaringan (SSID) Wi-Fi dan *password* yang digunakan pada bagian awal kode program.

   b. Memasukkan skrip HTML dan JavaScript ke dalam variabel C++ untuk tampilan web.

   c. Mengatur fungsi *handler* WebSocket di Arduino IDE menggunakan *library* pendukung, termasuk perintah ws.cleanupClients() di void loop agar memori RAM ESP8266 tidak penuh saat tab browser klien ditutup.

d. Melakukan *upload* program ke NodeMCU.

B. Hasil dan Pembahasan

Berdasarkan praktikum yang telah dilakukan, sistem berhasil berjalan dengan baik. Berikut adalah penjabaran hasil pengujiannya:

1. **Koneksi Jaringan:** Setelah program diunggah, NodeMCU berhasil terhubung ke jaringan Wi-Fi yang sudah disetel dengan nama SSID "MACHMUDALHASAN" dan *password* "Dodol Garut". Pada Serial Monitor, terlihat bahwa perangkat mendapatkan IP Address lokal yaitu 192.168.137.212. IP ini yang kemudian diakses menggunakan *browser* pada laptop.

2. **Pembacaan Sensor dan Tampilan Web:** Ada sedikit penyesuaian dari modul asli. Pada kodingan, tipe sensor diubah menyesuaikan perangkat keras yang dipakai, yaitu menjadi DHT dht(dhtPin, DHT22). Antarmuka web di laptop juga berhasil menampilkan data suhu "30.30 Celcius" beserta data tambahan berupa kelembapan "87.00 %". Ini menunjukkan bahwa pemisahan data JSON dari mikrokontroler berhasil dibaca oleh JavaScript di halaman web.

3. **Pemantauan WebSocket:** Aktivitas *client-server* berjalan lancar. Pada Serial Monitor, sistem merespons setiap kali halaman web dibuka atau ditutup dengan mencetak log seperti Client Websocket #1 terhubung, Client Websocket #2 terhubung, dan Client Websocket #2 terputus.

4. **Pengujian Sinkronisasi:** Seluruh perangkat fisik (LED, tombol, sensor) pada breadboard telah terangkai sesuai instruksi. Saat push button di breadboard ditekan, lampu fisik menyala dan tombol interaktif pada web juga langsung berubah otomatis menjadi "Turn ON". Perubahan ini terjadi secara instan tanpa adanya proses *loading* atau memuat ulang halaman browser, membuktikan bahwa komunikasi dua arah bekerja sempurna.

C. Kesimpulan

Dari praktikum ini dapat disimpulkan bahwa protokol WebSocket jauh lebih efektif dibandingkan komunikasi HTTP biasa untuk membuat dashboard IoT yang interaktif. Selain itu, penggunaan teknik pemrograman *non-blocking* dengan fungsi millis() sangat krusial. Jika tetap menggunakan delay(), sistem pasti akan mengalami *lag* saat membaca data dari sensor DHT, yang mengakibatkan respons penekanan tombol fisik menjadi terlambat atau bahkan tidak terbaca sama sekali. Sistem *Smart Room* sederhana ini sudah berhasil memenuhi seluruh target skenario sinkronisasi *real-time*

4.6 Latihan

1. Latihan 1 (Teori Jaringan): Jelaskan dengan kata-kata Anda sendiri, mengapa teknologi koneksi WebSockets jauh lebih hemat konsumsi data/bandwidth internet dan tidak membuat layar berkedip jika dibandingkan dengan metode Meta Refresh HTTP HTML yang Anda gunakan pada Tugas Praktikum Modul 3?

Berdasarkan praktikum pada Modul 4, teknologi WebSockets jauh lebih hemat bandwidth internet dan tidak membuat layar berkedip karena adanya perbedaan arsitektur komunikasi jika dibandingkan dengan HTTP biasa. Pada metode HTTP yang menggunakan Meta Refresh, browser dipaksa untuk meminta dan memuat ulang (reload) seluruh kerangka halaman web HTML secara terus-menerus dari awal. Proses memuat ulang seluruh elemen web hanya untuk memperbarui satu data kecil inilah yang membuat layar sering terlihat berkedip (flickering) dan memakan banyak kuota data jaringan.

Sedangkan pada teknologi WebSockets, koneksi yang terbangun antara browser dan mikrokontroler bersifat persisten (terus terbuka) dan dua arah. Halaman antarmuka HTML hanya perlu dimuat satu kali saat web pertama kali dibuka. Setelah itu, jika ada perubahan data seperti nilai suhu, mikrokontroler hanya perlu mendorong (push) paket data kecil berformat JSON. Script JavaScript di sisi klien kemudian menangkap paket tersebut dan hanya memperbarui angkanya saja pada elemen HTML yang dituju tanpa perlu me-refresh keseluruhan halaman. Hal ini membuat pembaruan antarmuka menjadi sangat responsif secara real-time dan jauh lebih efisien dalam penggunaan bandwidth.

2. Latihan 2 (Analisis Waktu/Timer): Apa bahaya sistematis yang akan terjadi pada komunikasi WebSockets (reaksi antara browser dan NodeMCU) jika Anda bersikeras nekat memanggil sintaks delay(3000) di dalam siklus fungsi void loop() pada draf arsitektur kompleks ini?

Berdasarkan analisis sistem pada Latihan 2, bahaya utama menggunakan fungsi delay(3000) di dalam void loop() pada arsitektur WebSockets adalah terjadinya blocking atau pembekuan proses secara total pada mikrokontroler. Ketika sintaks delay dipanggil, CPU NodeMCU akan berhenti bekerja dan menghentikan pengeksekusian baris kode lainnya selama 3 detik penuh. Akibatnya, jika pengguna mencoba menekan tombol kontrol di antarmuka web atau menekan sakelar fisik tepat saat masa jeda tersebut sedang berlangsung, sistem tidak akan bisa merespons perintah tersebut seketika. Hal ini akan membuat perangkat terasa sangat lambat (lag) atau bahkan mengabaikan input dari pengguna sama sekali. Selain merusak responsivitas antarmuka alat, berhentinya operasi CPU secara paksa ini juga sangat rentan merusak stabilitas koneksi jaringan. Protokol komunikasi WebSockets membutuhkan aliran

pertukaran data yang aktif agar terowongan koneksinya tetap terbuka. Jika NodeMCU dibiarkan "tertidur" selama 3 detik berulang kali karena fungsi delay, browser bisa mengira perangkat NodeMCU mengalami timeout atau terputus secara sepihak, yang berujung pada terputusnya koneksi WebSocket itu sendiri. Itulah sebabnya pada sistem IoT yang kompleks dan menuntut respons real-time, kita wajib menggunakan fungsi millis() sebagai pengukur waktu non-blocking agar mikrokontroler bisa terus bekerja memantau jaringan tanpa hambatan.

3. Latihan 3 (Modifikasi Front-End): Analisislah fungsi JavaScript pada blok // Merombak UI Tombol & Teks sesuai status hardware. Nama ID elemen HTML () apa saja yang menjadi target injeksi/pengubahan teks oleh script yang ditulis oleh Anda?

Berdasarkan analisis pada blok fungsi JavaScript bagian // Merombak UI Tombol & Teks sesuai status hardware, terdapat dua elemen HTML yang menjadi target injeksi atau pengubahan properti oleh script. Pertama, elemen dengan atribut id="toggleBtn", yang merupakan sebuah tombol. Elemen ini ditargetkan untuk mengubah teks labelnya (antara "Turn ON" atau "Turn OFF") serta mengganti kelas warnanya (melalui properti className) agar visual tombol menyesuaikan status nyala lampu. Kedua, elemen dengan atribut id="ledStatus" (sebuah elemen), yang menjadi target injeksi untuk mengubah teks keterangannya secara langsung menjadi tulisan "ON" atau "OFF" sesuai dengan paket data yang dikirimkan oleh NodeMCU.

4. Latihan 4 (Pengembangan Telemetri Sensor - Kelembapan): Pada Praktikum 1, antarmuka web hanya menyajikan data suhu (temperature). Modifikasilah program agar sistem dapat membaca dan menampilkan data kelembapan (humidity) secara real-time via WebSockets:

   i. Sisi Back-End (C++): Buat variabel penampung (misal: String currentHum = "--";). Baca nilai kelembapan menggunakan fungsi dht.readHumidity() di dalam siklus timer non-blocking millis(). Perbarui fungsi notifyClients() agar menyisipkan data kelembapan ke dalam paket JSON yang dikirim ke browser (contoh format: {"led":"...", "suhu":"...", "hum":"..."}).
   ii. Sisi Front-End (HTML & JavaScript): Tambahkan elemen kartu ( ) baru pada kerangka HTML sebagai penampil kelembapan (misal: <h2>Kelembapan: -- %</h2>). Kemudian, modifikasi fungsi JavaScript onMessage(event) agar mengekstrak data kelembapan dari objek JSON dan memperbarui teks pada elemen humValue.
   Tuliskan baris kode modifikasi Anda (pada sisi HTML, JavaScript, dan C++)!

Pada Latihan 4, sistem telemetri dikembangkan agar tidak hanya membaca suhu, tetapi juga menampilkan data kelembapan (humidity) secara real-time via WebSockets. Proses modifikasi ini dibagi menjadi dua bagian utama: penyesuaian di sisi server mikrokontroler (Back-End) dan pembaruan antarmuka web (Front-End).

i. Modifikasi Sisi Back-End (C++)

Pertama, saya menambahkan variabel global baru bertipe String, yaitu String currentHum = "--";, untuk menyimpan data kelembapan sementara. Selanjutnya, pada siklus pembacaan sensor di dalam void loop() (yang menggunakan timer non-blocking millis()), saya memanggil fungsi dht.readHumidity() dan menyimpan hasilnya ke dalam variabel tersebut. Terakhir, fungsi perakit pesan notifyClients() dirombak agar ikut menyisipkan variabel kelembapan ke dalam kerangka data JSON.

Berikut adalah potongan baris kode C++ yang dimodifikasi:

```
// Di bagian atas (Deklarasi Variabel)
String currentHum = "--";

// Di dalam void loop() -> Blok pembacaan timer millis()
float h = dht.readHumidity();
if(!isnan(t) && !isnan(h)) {
  currentTemp = String(t, 1);
  currentHum = String(h, 1);
  notifyClients();
}

// Di dalam fungsi notifyClients()
void notifyClients() {
  String jsonString = "{\"led\":\"" + String(ledState ? 1 : 0) + "\", ";
  jsonString += "\"suhu\":\"" + currentTemp + "\", ";
  jsonString += "\"hum\":\"" + currentHum + "\"}"; // Menyisipkan key 'hum'
  ws.textAll(jsonString);
}
```

ii. Modifikasi Sisi Front-End (HTML & JavaScript)

Agar data baru tersebut dapat dilihat oleh pengguna, saya menambahkan sebuah elemen "kartu" ( ) baru di dalam struktur HTML yang memiliki sebuah elemen rentang . Setelah antarmuka visualnya siap, saya memodifikasi fungsi pendengar onMessage(event) pada blok JavaScript agar mampu mengekstrak nilai kunci hum dari paket JSON yang datang, lalu menyuntikkan nilai tersebut langsung ke dalam ID elemen HTML yang baru saja dibuat.

Berikut adalah potongan baris kode HTML dan JavaScript yang dimodifikasi:

```
<!-- Penambahan pada blok <body> HTML -->
<div class="card">
  <h2>Kelembapan: <span id="humValue">--</span> %</h2>
</div>

<script>
// Penambahan pada fungsi onMessage(event) di JavaScript
function onMessage(event) {
  var dataObj = JSON.parse(event.data);

  // Logika ekstraksi yang ditambahkan:
  if(dataObj.hum !== undefined) {
    document.getElementById('humValue').innerHTML = dataObj.hum;
  }

  // ... (kode lainnya tetap sama) ...
}
</script>
```